

Aufgaben zum Aufgabenblock 4 - C-Teil für TI und ITSEC

1. Einfache Ein- und Ausgaben

1.1 Ausgaben

Code Beispiel:

```
#include <stdio.h>

int main() {
    char account[50] = "Girokonto";
    char bank[50] = "Sparkasse";
    int bank_code = 512700;
    double balance = 4314.34;
    double interest = 1.99;
    char currency[50] = "EUR";

    // Your Code

    return 0;
}
```

1.1.1 Ausgaben mit printf

Verwende die Funktion `printf()` um eine Ausgabe zu erzeugen, die wie folgt aussieht:

```
Bank: Sparkasse
Konto: Girokonto
Kontostand: 4314.34 EUR
Zinssatz: 1.99 %
```

1.1.2 Formatierte Ausgabe mit printf

Erstelle eine extra Variable mit einem passenden Datentyp und speichere darin folgende Berechnung:

```
(balance * interest) / 12
```

Als Ausgabe sollte folgendes ausgegeben werden:

Monatliche Zinsen: 7.17 EUR

1.2 Eingaben

Schreibe ein Programm, das folgende Eingaben und Ausgaben erzeugt:

```
Preisrechner  
Produktname: Apfel  
Anzahl: 5  
Einzelpreis: 0.45 EUR  
Gesamtpreis: 2.25 EUR
```

2. Funktionen, Parameter und Kommandozeilen-Parameter

2.1 Kommandozeilen Parameter

Erstelle ein Programm, dass folgende Ausgabe erzeugt. Erstelle hierfuer eine Datei mit Namen und rufe es am besten wie folgt auf:

```
./programm argument1 argument2 argument3
```

```
Programm-Name: programm  
1. Argument: argument1  
2. Argument: argument2  
3. Argument: argument3
```

2.2 Funktionen

2.2.1 Einfache Funktionen und Funktionen in Funktionen

Erstelle eine Funktion in dem angegebenen Bereich. Die Funktion soll eine formattierte Ausgabe erzeugen. Hierfuer sollen alle benoetigten Informationen als Parameter uebergeben werden.

- Optional: erstelle eine Funktion `separator()` welche eine die Striche printed. Die Funktion kann auch innerhalb einer anderen Funktion, also der `formatted_output` aufgerufen werden

```
// Your Code  
// func-name: formatted_output()
```

```

int main () {
    char first_name[50] = "Max";
    char last_name[50] = "Mustermann";
    int matrikel_nr = 14434;
    int semester = 20;
    double ects = 33.33;

    // here goes the func-call

    return 0;
}

```

Die Ausgabe sollte so aussehen:

```

-----
Student      : Max Mustermann
Matrikel-Nummer: 14434
Semester     : 3
ECT          : 33.33
-----

```

2.2.2 Funktionen mit Rückgabewert

Erstelle einen Annuitätenrechner. Verwende wenn noetig math.h (Gerne auch versuchen, eine eigene Pow-Funktion zu schreiben).

```

#include <stdio.h>
#include <math.h>

// optional func calc_pow with return value

// func with return
// double calc_annuity(...)

int main() {

    double amounts[5] = {
        5000.0,
        6000.0,
        7000.0,
        8000.0,
        9000.0
    }
}

```

```

};
double i_rates[5] = { 5.0, 6.0, 7.0, 8.0, 9.0 };
int years[5] = { 1, 3, 5, 7, 10 };
// your code

return 0;
}

```

Die Formel lautet:

$$A = K \cdot \frac{(1+p)^n - 1}{p \cdot (1+p)^n}$$

A: Annuität

K: Betrag

p: Zinssatz (0.05 für 5%)

n: Anzahl Jahre

Ausgabe:

```

Kreditbetrag: 10000.00 EUR
Zinssatz: 5.00 %
Laufzeit: 10 Jahre
Jaehrliche Annuitaet: 1295.05 EUR
Monatliche Annuitaet: 107.92 EUR

```

```

Kreditbetrag: 10000.00 EUR
Zinssatz: 5.00 %
Laufzeit: 10 Jahre
Jaehrliche Annuitaet: 1295.05 EUR
Monatliche Annuitaet: 107.92 EUR

```

```

.
.
.

```

2.2.3 Filter and Replace & Header-Dateien

Es soll ein Programm mit Funktionen geschrieben werden, welches eine Kette von Zeichen einliest, sowie ein Spezialzeichen, mit dem Ziffern ersetzt werden sollen.

Teile das Programm in Header-Datei, Implementierungs-Datei und main-Datei auf.

Anforderungen:

- Sonderzeichen einlesen und abspeichern
- Zeichenkette einselen und abspeichern (array/buffer)
- find-and-replace-Funktion schreiben
 - sucht alle Zahlen und ersetzt diese mit dem Sonderzeichen
 - sucht nach Kleinbuchstaben und ersetzt diese mit Grossbuchstaben
 - sucht nach Grossbuchstaben und ersetzt diese mit Kleinbuchstaben
 - Optional weitere Zeichen finden und ersetzen
- Ausgabe:
 - originale Zeichenkette
 - neue Zeichenkette

3. structs, Pointer, malloc & free

Es soll ein Basic Game-Setup für das Spiel Space Invaders erstellt werden. Hierfür werden 3 verschiedene Entitäten benötigt, Starship, Enemy, Bullet. Zudem sollen simple Standard-Funktionen wie create, move, fire erstellt werden.

Anforderungen Entities:

- structs fuer Enemy, Starhip und Bullet
 - es soll ein Name vergeben werden an alles ausser der Bullet
 - es sollen x- und y-Werte fuer die Position abgespeichert werden koennen
 - zudem soll es einen status alive/lives/hitpoints und active geben
- es soll eine `create_enemy`-Funktion geben, welche
 - mittels malloc einen Enemy erstellt und zurueckgibt
 - bei der Erstellung (Standard-) Werte setzt
- es soll eine `move_enemy` - Function erstellt werden, die
 - eine Referenz auf einen Enemy bekommt und neue Positions
 - Werte. Die Funktion soll diese entsprechend draufaddieren
- das gleiche mit Bullet
 - zudem soll fuer Bullets noch eine `fire_bullet`-Funktion erstellt werden, welche den Status der Bullet auf active

setzt

Schreibe den Code wieder getrennt in Header- und Source-Dateien und erstelle ein main Programm.

Anforderungen Main Programm:

- es soll mindestens ein Enemy erstellt werden
- in einer variable gespeichert werden
- die startposition ausgegeben werden
- einmal bewegt werden
- die neue Position ausgegeben werden
- am Ende mit free der Speicherplatz wieder freigegeben werden